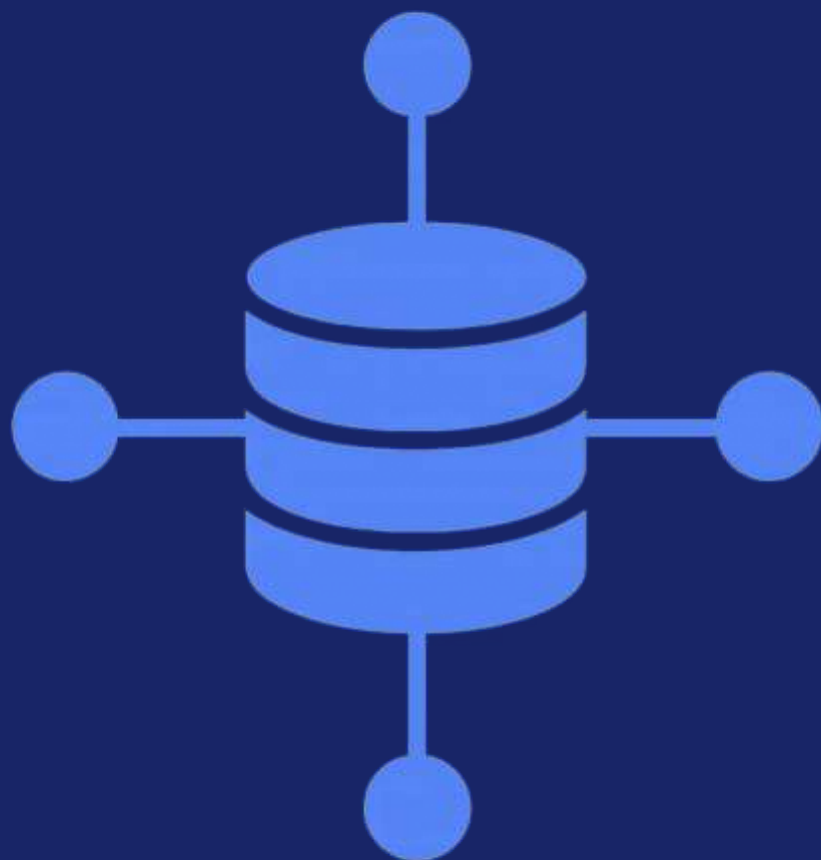




GIORNO 3

Oracle SQL – Join e Subquery

Oggi impariamo a **unire dati da più tabelle** e a costruire query complesse con subquery. Al termine della giornata sarai in grado di scrivere query che collegano tabelle relazionali e di utilizzare subquery per condizioni dinamiche.

- Unire dati da più tabelle con JOIN
- Comprendere le relazioni tra tabelle
- Scrivere subquery efficaci



Prerequisiti: SELECT, WHERE, ORDER BY, GROUP BY, funzioni aggregate –
Giorno 2

Perché Servono le JOIN?

In un database relazionale i dati sono **distribuiti su più tabelle** per evitare ridondanza e garantire integrità. Le JOIN permettono di ricostruire le informazioni complete al momento della query.

Dati Distribuiti

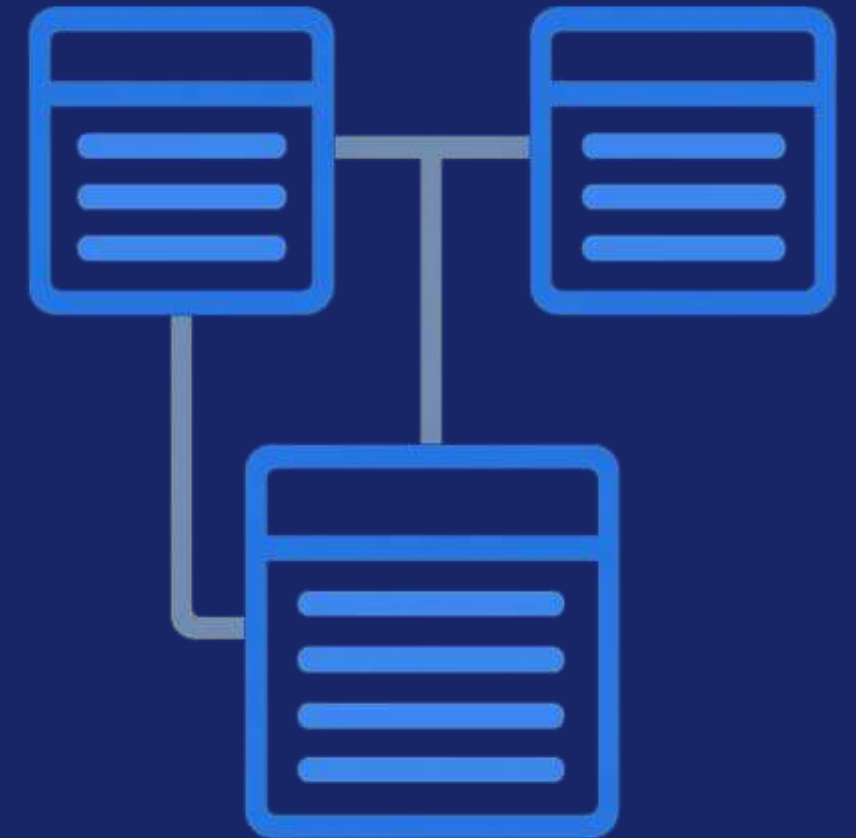
Ogni tabella contiene informazioni specifiche. **employees** ha i dati anagrafici, **departments** i nomi dei reparti.

Nessuna Duplicazione

Il nome del reparto è memorizzato una sola volta in **departments**, non ripetuto in ogni riga di **employees**.

Chiavi di Collegamento

Le tabelle si collegano tramite **chiavi esterne**: `department_id` in **employees** punta a **departments**.



JOIN con WHERE – INNER JOIN Implicita

Sintassi Base

Si elencano le tabelle nella **FROM** e si aggiunge la condizione di join nella **WHERE**.

```
SELECT e.first_name,  
       d.department_name  
FROM   employees e,  
       departments d  
WHERE  e.department_id  
       = d.department_id;
```

Come Funziona

- **FROM**: si elencano le tabelle coinvolte
- **WHERE**: si specifica la condizione di uguaglianza tra le chiavi
- Solo le righe con corrispondenza vengono restituite
- I dipartimenti senza dipendenti **non compaiono**

📌 Questa è una **INNER JOIN implicita**: restituisce solo le righe con corrispondenza in entrambe le tabelle.

JOIN tra Più Tabelle

È possibile unire **tre o più tabelle** aggiungendo condizioni nella clausola `WHERE`. Ogni nuova tabella richiede una condizione di join aggiuntiva.

```
SELECT e.first_name,  
       d.department_name,  
       j.job_title  
FROM   employees e,  
       departments d,  
       jobs j  
WHERE  e.department_id = d.department_id  
       AND e.job_id      = j.job_id;
```

Prima Condizione

`e.department_id = d.department_id`
— collega **employees** a **departments**

Seconda Condizione

`e.job_id = j.job_id` — collega
employees a **jobs**

Risultato

Ogni riga contiene nome del dipendente,
reparto e mansione in un'unica query

⚠️ Attenzione: Prodotto Cartesiano

Cosa Succede Senza WHERE

Se si omette la condizione di join, Oracle restituisce **tutte le combinazioni possibili** tra le righe delle tabelle.

```
SELECT *  
FROM   employees,  
       departments;
```

⚠️ **Risultato:** 107 dipendenti × 27 reparti = **2.889 righe** — quasi tutte errate!

Come Evitarlo

- Aggiungere **sempre** la condizione di join nella **WHERE**
- Verificare il numero di righe restituite rispetto alle aspettative
- Usare alias di tabella per chiarezza (e., d.)

📘 Il prodotto cartesiano è uno degli errori più comuni nei principianti. Controlla sempre le condizioni **WHERE**.

Simulare LEFT JOIN – Concetto

Le join con `WHERE` eseguono solo **INNER JOIN**: le righe senza corrispondenza vengono **escluse**. Per includerle tutte servono tecniche avanzate.

INNER JOIN (WHERE)

Solo dipendenti **con reparto assegnato**. I dipendenti senza `department_id` non compaiono.

LEFT JOIN (obiettivo)

Tutti i dipendenti, anche quelli senza reparto. Il campo reparto sarà `NULL` se non assegnato.

📌 In Oracle la **LEFT OUTER JOIN** si scrive con la sintassi `(+)` o con `LEFT JOIN` esplicito (SQL:1999). Approfondiremo nelle giornate avanzate.

Introduzione alle Subquery

Una **subquery** è una query nidificata all'interno di un'altra query. Viene eseguita per prima e il suo risultato viene usato dalla query esterna.

Esempio: Salario Superiore alla Media

```
SELECT *  
FROM employees  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
);
```

Come Funziona

1. La subquery calcola la **media dei salari**
2. Il valore viene passato alla query esterna
3. La query esterna filtra i dipendenti con salario **superiore**

La subquery restituisce un **singolo valore scalare**.

Subquery con IN

Quando la subquery restituisce **più valori**, si usa l'operatore **IN** per verificare l'appartenenza a un insieme dinamico.

```
SELECT first_name, last_name
FROM   employees
WHERE  department_id IN (
    SELECT department_id
    FROM   departments
    WHERE  location = 'Roma'
);
```

1 Subquery interna

Restituisce tutti i `department_id` dei reparti con sede a **Roma** (può restituire più righe)

2 Query esterna

Filtra i dipendenti il cui `department_id` è presente nell'elenco restituito dalla subquery

3 Vantaggio

Nessun hardcoded: se i reparti di Roma cambiano, la query si **aggiorna automaticamente**



Esercitazioni Pratiche

Mettiamo in pratica i concetti appresi. Completa le seguenti query sul dataset **employees**, **departments** e **jobs**.

1

Join Base

Unisci **employees** e **departments**: mostra nome, cognome e nome del reparto.

2

Join su 3 Tabelle

Aggiungi la tabella **jobs**: mostra nome, reparto e **job_title** per ogni dipendente.

3

Subquery con AVG

Seleziona i dipendenti con salario superiore alla **media aziendale**.

4

Subquery con IN

Elenco dipendenti nei reparti con sede a **'Roma'** o **'London'**.

Riepilogo – Giorno 3

Concetti chiave appresi oggi e punti di attenzione per scrivere query corrette ed efficienti.



JOIN con WHERE

Unisci tabelle con condizioni di uguaglianza nella clausola **WHERE**
(INNER JOIN implicita)



Condizioni Multiple

Unisci tre o più tabelle aggiungendo condizioni con **AND** nella **WHERE**



Subquery

Query nidificate per condizioni dinamiche: valori scalari con confronto, liste con **IN**



Errori Comuni

Prodotto cartesiano (WHERE mancante), join incomplete, confusione tra INNER e LEFT JOIN



Domande? Approfondiamo insieme i casi pratici. Prossima lezione: sintassi JOIN esplicita e OUTER JOIN.